

Esercizio B1

È possibile risolvere questo esercizio costruendo un'opportuna rete di flusso $G = (V, E, s, p, c)$. Per quanto riguarda l'insieme V :

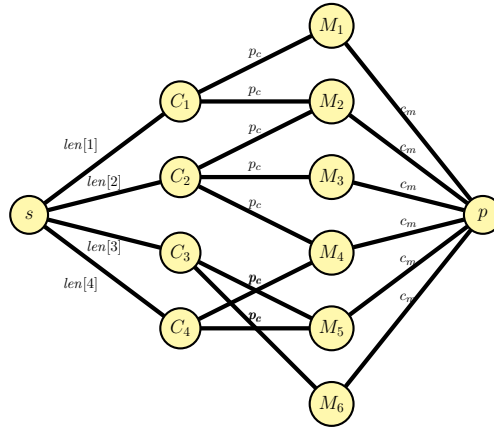
- Si creano un nodo sorgente s e un nodo pozzo p ;
- Si creano c nodi, uno per ogni cantiere;
- Si creano m nodi, uno per ogni mese.

Per quanto riguarda l'insieme E :

- Si aggiunge un arco fra la sorgente e ogni cantiere i , con capacità $len[i]$ (ad indicare che il progetto i richiede $len[i]$ mesi/persona).
- Si aggiunge un arco fra ogni progetto i e ogni mese incluso fra $start[i]$ e $end[i]$, con capacità p_c , ad indicare che ogni cantiere può occupare al massimo p_c persone durante un mese;
- Si aggiunge un arco fra ogni mese j e il pozzo, con capacità p_m , ad indicare che ogni mese sono disponibili al più p_m persone.

Sia $L = \sum_i^c len[i]$, ovvero il numero totale di mesi/persona che devono essere impegnati dal progetto. Si calcoli il flusso massimo sulla rete di flusso; se il flusso massimo è pari a L , è possibile completare tutti i cantieri seguendo le regole. Infatti, ogni cantiere riceverà il numero corretto di mesi/persona (primo strato); ad ogni cantiere non potranno lavorare più di p_c persone per mese (secondo strato); in ogni mese non potranno lavorare più di p_m persone (terzo strato).

Poiché $|V| = c + m + 2$ e $E = c + m + O(cm)$, e poiché il flusso è limitato superiormente da L , l'algoritmo avrà costo $O((|V| + |E|)L) = O(cmL)$ secondo il limite superiore di Ford-Fulkerson, $O((c + m)(c^2m^2))$ secondo Edmonds e Karp.



Esercizio B2

Si può risolvere il problema utilizzando programmazione dinamica.

Soluzione $\Theta(n^3)$

La domanda su pseudo-polinomialità vi potrebbe indurre a pensare che debba essere risolto con una tabella di programmazione dinamica a tre indici: sia $DP[i][j][k]$ uguale a **true** se e solo se la sottostringa $S[i \dots j]$ è k -palindroma. La tabella DP può essere calcolata nel modo seguente:

$$DP[i][j][k] = \begin{cases} \text{false} & k < 0 \\ \text{true} & j \leq i \text{ and } k \geq 0 \\ S[i] = S[j] \wedge DP[i+1][j-1][k] \text{ or } & \\ DP[i+1][j][k-1] \text{ or } & \\ DP[i][j+1][k-1] & i < j \text{ and } k \geq 0 \end{cases}$$

- Caso 1: se ho effettuato troppe rimozioni e $k < 0$, restituisco **false**.
- Caso 2: Nel caso in cui la stringa sia composta da 1 carattere ($j = i$) o 0 caratteri ($j < i$) e $k \geq 0$, allora la stringa è palindroma.
- Caso 3: Altrimenti, possono darsi tre sottocasi
 - I caratteri $S[i], S[j]$ sono uguali; la stringa $S[i \dots j]$ è k -palindroma se $S[i + 1 \dots j - 1]$ è k -palindroma;
 - Altrimenti, si elimini il carattere i -esimo e si verifichi se $S[i + 1 \dots j]$ è $k - 1$ -palindroma;
 - Oppure, si elimini il carattere j -esimo e si verifichi se $S[i \dots j - 1]$ è $k - 1$ -palindroma.

L'algoritmo può essere scritto utilizzando memoization. Utilizziamo il valore -1 per indicare una casella non ancora calcolata, 0 e 1 per indicare vero e falso, rispettivamente.

boolean k-palindrome(ITEM[] S , **int** n , **int** k)

```

DP = new int[1...n][1...n][1...k]
k = min(n, k)
for i = 1 to n do
  for j = 1 to n do
    for l = 1 to k do
      DP[i][j][l] = -1
return kpalRec(S, 1, n, k) == 1

```

int kpalRec(ITEM[] S , **int** i , **int** j , **int** k)

```

if k < 0 then
  return 0
if k ≥ 0 and j ≤ i then
  return 1
if DP[i][j][k] < 0 then
  DP[i][j][k] = (S[i] == S[j] and kpalRec(S, i + 1, j - 1, k)) or
                kpalRec(S, i + 1, j, k - 1) or
                kpalRec(S, i, j - 1, k - 1)
return DP[i][j][k]

```

La complessità è $O(kn^2)$; essendo k limitato superiormente da n , possiamo dire che l'algoritmo è $O(n^3)$, il che lo rende polinomiale.

Soluzione $\Theta(n^2)$

Ma si può fare meglio di così: sia $DP[i][j]$ il numero minimo di caratteri che devono essere cancellati per rendere la stringa palindroma. La tabella DP può essere calcolata nel modo seguente:

$$DP[i][j] = \begin{cases} 0 & j \leq i \\ \min(DP[i+1][j], DP[i][j-1]) + 1 & i < j \wedge S[i] \neq S[j] \\ DP[i+1][j-1] & i < j \wedge S[i] = S[j] \end{cases}$$

- Caso 1: Nel caso in cui la stringa sia composta da 1 carattere ($j = i$) o 0 caratteri ($j < i$), allora la stringa è palindroma e il numero minimo di cancellazioni è 0.
- Casi 2,3: Altrimenti, possono darsi due sottocasi

- I caratteri $S[i], S[j]$ sono uguali; il numero minimo di caratteri è pari al minimo fra il numero minimo di caratteri per rendere palindroma $S[i \dots j - 1]$ o $S[i + 1 \dots j]$.
- I caratteri $S[i], S[j]$ sono uguali; il numero minimo di caratteri da cancellare per rendere palindroma la stringa $S[i \dots j]$ è pari al numero di caratteri per rendere palindroma la stringa $S[i + 1 \dots j - 1]$;

```
boolean k-palindrome(ITEM[] S, int n, int k)
```

```
    DP = new int[1...n][1...n]
    for i = 1 to n do
        for j = 1 to n do
            DP[i][j] = -1
    return kpalRec(S, 1, n) ≤ k
```

```
int kpalRec(ITEM[] S, int i, int j)
```

```
    if j ≤ i then
        return 0
    if DP[i][j] < 0 then
        if S[i] == S[j] then
            DP[i][j] = kpalRec(S, i + 1, j - 1)
        else
            DP[i][j] = min(kpalRec(S, i + 1, j), kpalRec(S, i, j - 1)) + 1
    return DP[i][j]
```

Questo algoritmo ha complessità $O(n^2)$.

Soluzione $O(2^n)$

Notate che mi capita spesso di vedere codice come questo:

```
int kpalRec(ITEM[] S, int i, int j)
```

```
    if j ≤ i then
        return 0
    if S[i] == S[j] then
        return kpalRec(S, i + 1, j - 1)
    else
        return min(kpalRec(S, i + 1, j), kpalRec(S, i, j - 1)) + 1
```

Notate che manca il controllo sul valore -1 necessario per la memoization? Questo trasforma la soluzione da polinomiale a superpolinomiale, perché lo stesso problema può essere calcolato più volte; è formalmente corretta, ma estremamente inefficiente. Per questo prende normalmente 60%.

Esercizio B3

Il problema corrisponde a identificare almeno un cammino hamiltoniano che inizia in un certo nodo, e si risolve tramite un algoritmo di backtrack.

L'algoritmo sfrutta il template degli algoritmi basati su backtrack visti a lezione. Utilizza un vettore *visited* per memorizzare i nodi già visitati nel cammino attuale; a differenza di una semplice visita, quando si effettua un'operazione di backtrack si cancella la visita dal vettore.

```
int[] safeLine(GRAPH G, int u)
```

```
    boolean[] visited = new int[1...G.n]  
    for i = 1 to n do  
        | visited[i] = false  
    int[] S = new int[1...G.n]  
    return safeLineRec(G, u, visited, S, 1)
```

```
int[] safeLineRec(GRAPH G, int u, boolean[] visited, int[] S, int i)
```

```
    S[i] = u  
    if i == G.n then  
        | return S  
    else  
        | visited[u] = true  
        | foreach v ∈ G.adj(u) : visited[v] == false do  
            | int[] res = safeLineRec(G, v, visited, S, i + 1)  
            | if res ≠ nil then  
                | return res  
        | visited[u] = false  
        | return nil
```

La complessità è $O(n!)$, in quanto potenzialmente bisogna considerare tutte le $(n - 1)!$ permutazioni di $n - 1$ nodi (il primo è fisso).